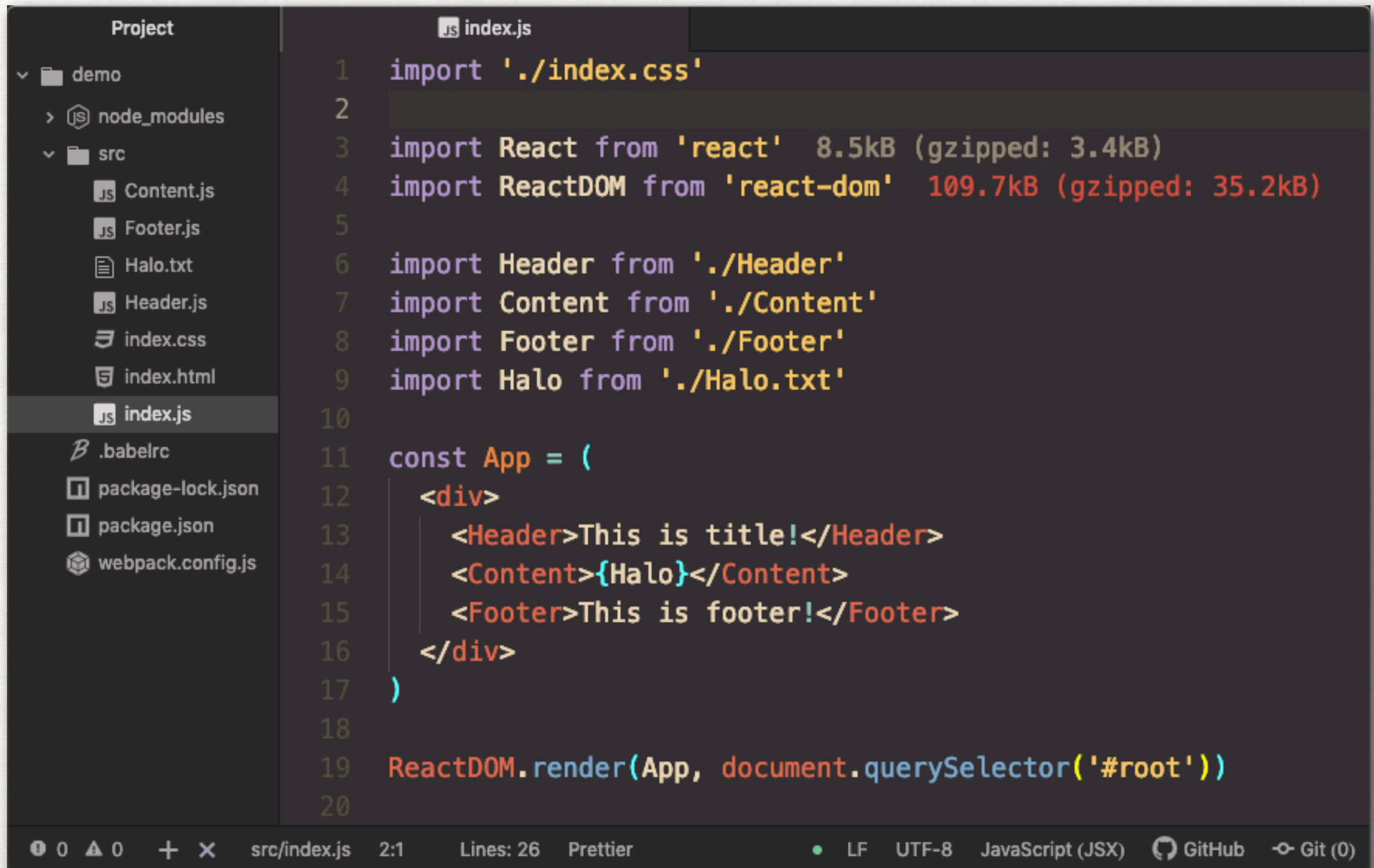


编译&构建 前端应用

BABEL



现代前端应用示例



The image shows a code editor interface with a project explorer on the left and a code editor on the right. The project explorer shows a directory structure with a 'demo' folder containing 'node_modules' and 'src'. The 'src' folder contains 'Content.js', 'Footer.js', 'Halo.txt', 'Header.js', 'index.css', 'index.html', and 'index.js'. The 'index.js' file is selected and its content is displayed in the code editor. The code is a JavaScript file using ES6 imports and JSX to render a simple application. It imports 'index.css', 'react', 'react-dom', 'Header', 'Content', 'Footer', and 'Halo.txt'. It then defines a constant 'App' which is a JSX element containing a header, content, and footer. Finally, it calls 'ReactDOM.render' to render the 'App' into the '#root' element of the document.

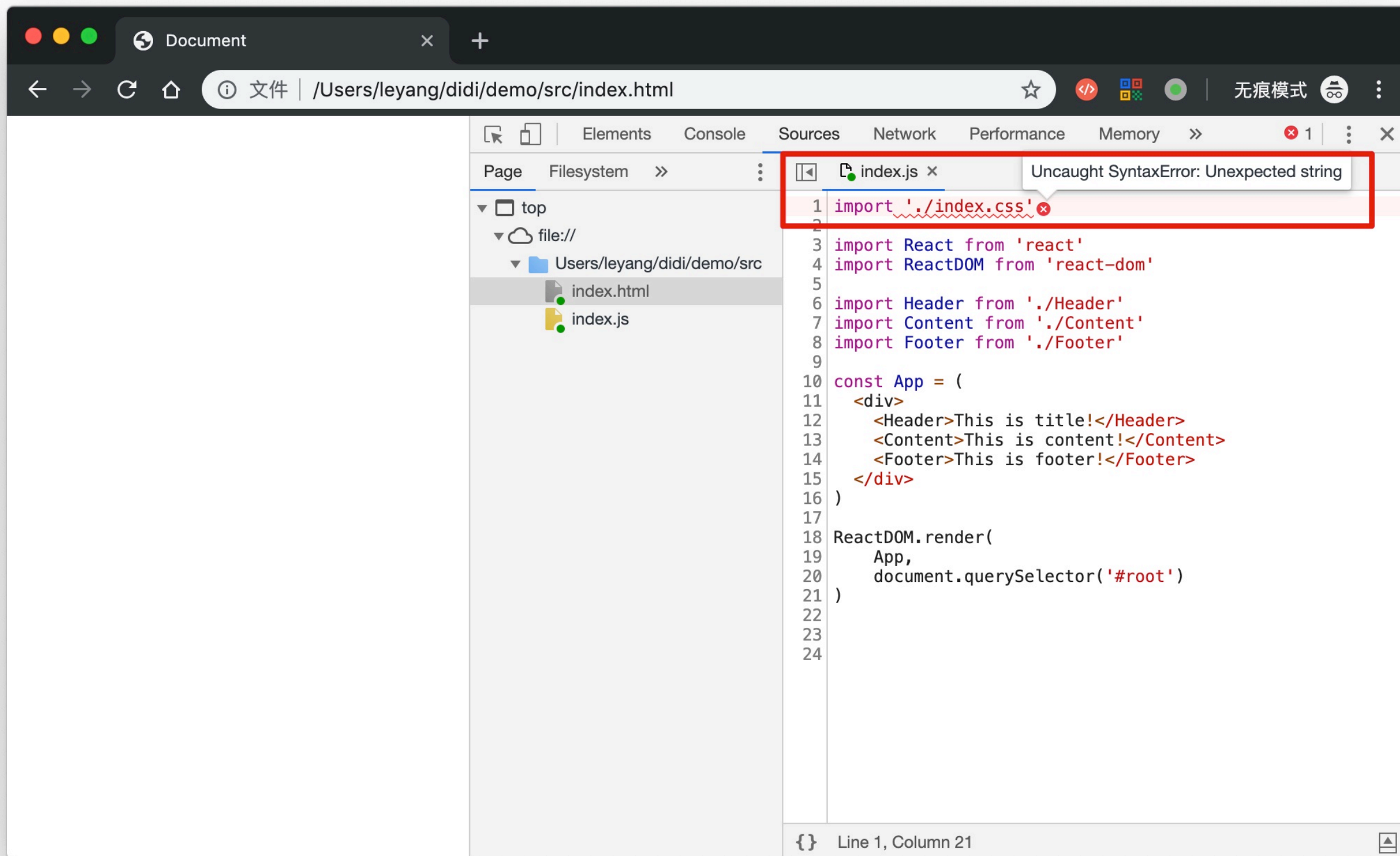
```
1 import './index.css'
2
3 import React from 'react' 8.5kB (gzipped: 3.4kB)
4 import ReactDOM from 'react-dom' 109.7kB (gzipped: 35.2kB)
5
6 import Header from './Header'
7 import Content from './Content'
8 import Footer from './Footer'
9 import Halo from './Halo.txt'
10
11 const App = (
12   <div>
13     <Header>This is title!</Header>
14     <Content>{Halo}</Content>
15     <Footer>This is footer!</Footer>
16   </div>
17 )
18
19 ReactDOM.render(App, document.querySelector('#root'))
20
```

Project

- demo
 - node_modules
 - src
 - Content.js
 - Footer.js
 - Halo.txt
 - Header.js
 - index.css
 - index.html
 - index.js**
 - .babelrc
 - package-lock.json
 - package.json
 - webpack.config.js

0 0 + x src/index.js 2:1 Lines: 26 Prettier LF UTF-8 JavaScript (JSX) GitHub Git (0)

直接引入



为什么报错?

- 缺少新语法支持
- 缺少模块化构建支持

那么，如何解决？

编译不支持的语法

The screenshot shows the Babel REPL interface. The left pane contains the input code, which is an ES6 module using `import` and `export` syntax. The right pane shows the output code, where Babel has transformed the imports into `require` calls and the module structure into a single function call to `ReactDOM.render`. The output code includes a `"use strict";` directive and uses `_react`, `_reactDom`, `_Header`, and `_Content` to represent the imported modules. A `_interopRequireDefault` function is also shown, which is a helper function used by Babel to handle default exports.

```
1 import './index.css'
2
3 import React from 'react'
4 import ReactDOM from 'react-dom'
5
6 import Header from './Header'
7 import Content from './Content'
8 import Footer from './Footer'
9
10 const App = (
11   <div>
12     <Header></Header>
13     <Content></Content>
14     <Footer></Footer>
15   </div>
16 )
17
18 ReactDOM.render(
19   App,
20   document.querySelector('#root')
21 )
22
```

```
1 "use strict";
2
3 require("./index.css");
4
5 var _react = _interopRequireDefault(require("react"));
6
7 var _reactDom = _interopRequireDefault(require("react-dom"));
8
9 var _Header = _interopRequireDefault(require("./Header"));
10
11 var _Content = _interopRequireDefault(require("./Content"));
12
13 var _Footer = _interopRequireDefault(require("./Footer"));
14
15 function _interopRequireDefault(obj) { return obj &&
16   obj.__esModule ? obj : { default: obj }; }
17
18 var App = _react.default.createElement("div", null,
19   _react.default.createElement(_Header.default, null),
20   _react.default.createElement(_Content.default, null),
21   _react.default.createElement(_Footer.default, null));
22
23 _reactDom.default.render(App, document.querySelector('#root'));
```


模块化构建入口

index.bundle.js

node_modules

react-dom

cjs

react-dom.development.js

react

cjs

react.development.js

scheduler

cjs

scheduler.development.js

style-loader/lib

prop-types

addStyles.js

utils.js

checkPropTypes.js

css-loader/dist/runtime

api.js

object-assign

index.js

src

Stat size: **2.55 KB**
Parsed size: **4.4 KB**
Gzipped size: **912 B**
Path: **./src**

JS 编译器

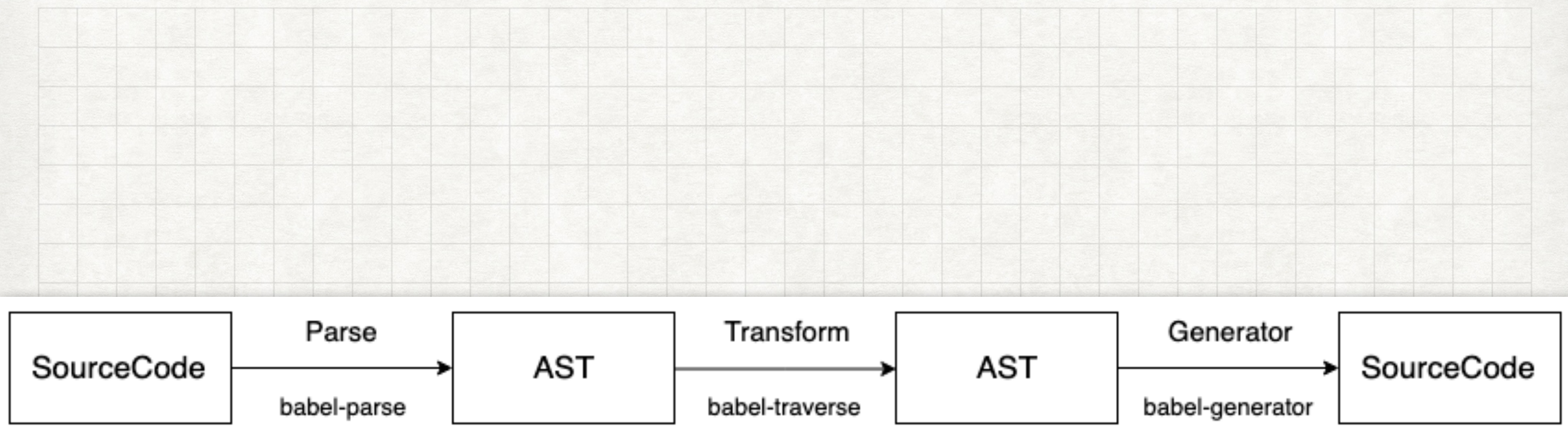
BABEL

核心概念

```
.babelrc
1 {
2   "presets": ["@babel/preset-env"],
3   "plugins": [
4     "@babel/plugin-transform-react-jsx"
5   ]
6 }
7
```

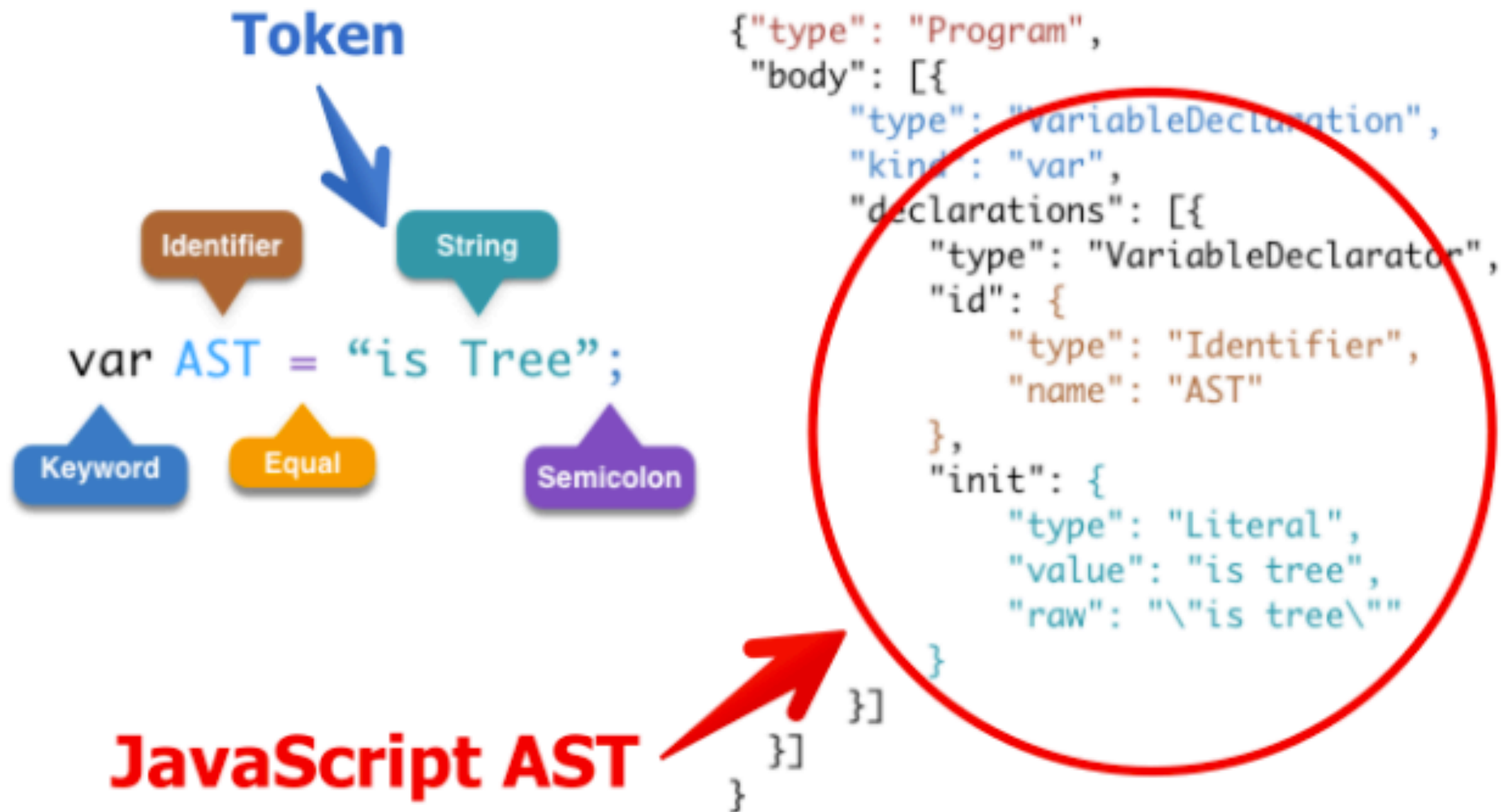
0 0 + x .babelrc 7:1 Lines: 7 Prettier • LF UTF-8 JSON GitHub Git (0) 1 update

核心流程



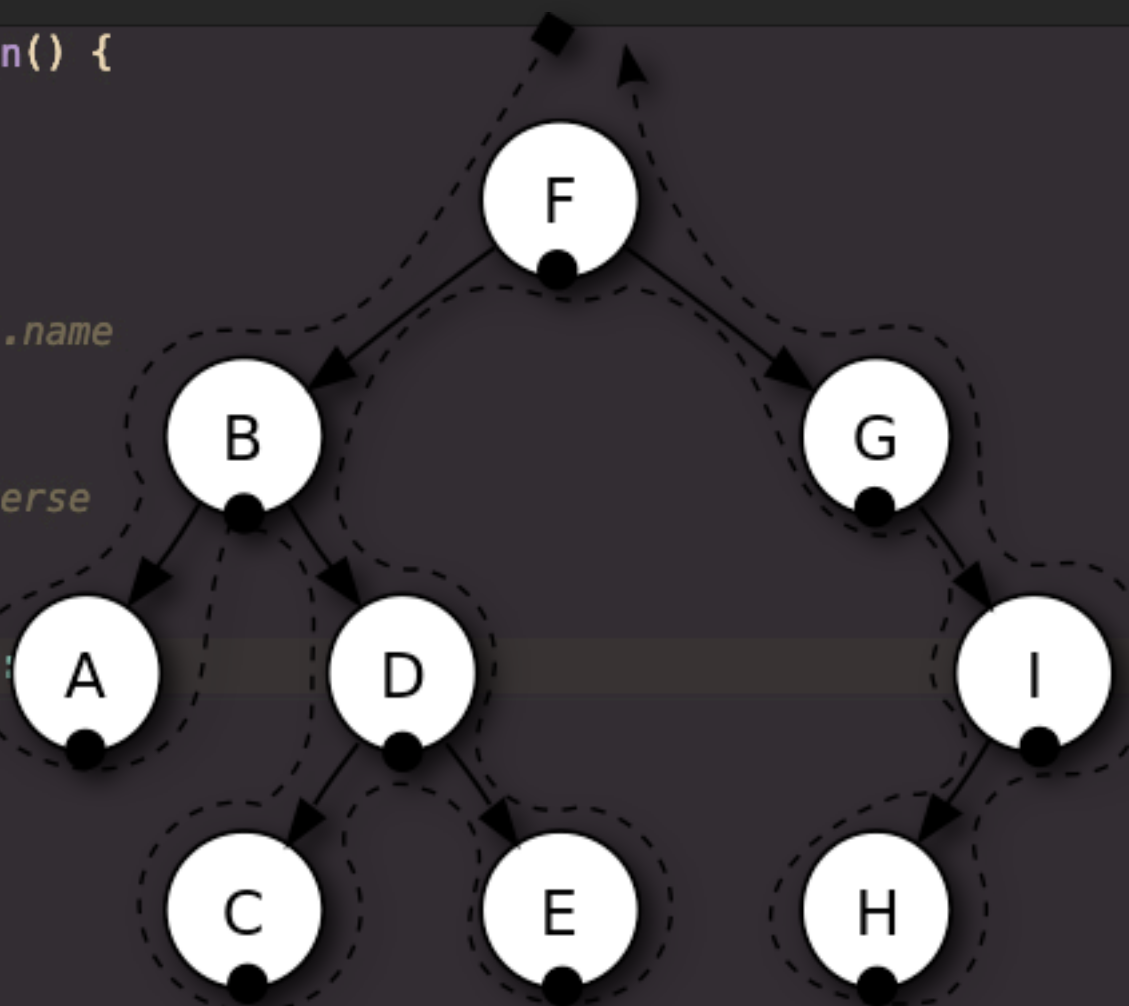
AST

抽象语法树



AST 遍历

```
1 export default function() {  
2   return {  
3     visitor: {  
4       Identifier: {  
5         entry(path) {  
6           // path.node.name  
7         }  
8         exist(path) {  
9           // path.traverse  
10        }  
11      },  
12      BinaryExpression:  
13    },  
14  };  
15 }  
16
```



插件示例

PLUGIN-TRANSFORM-MODULES-COMMONJS

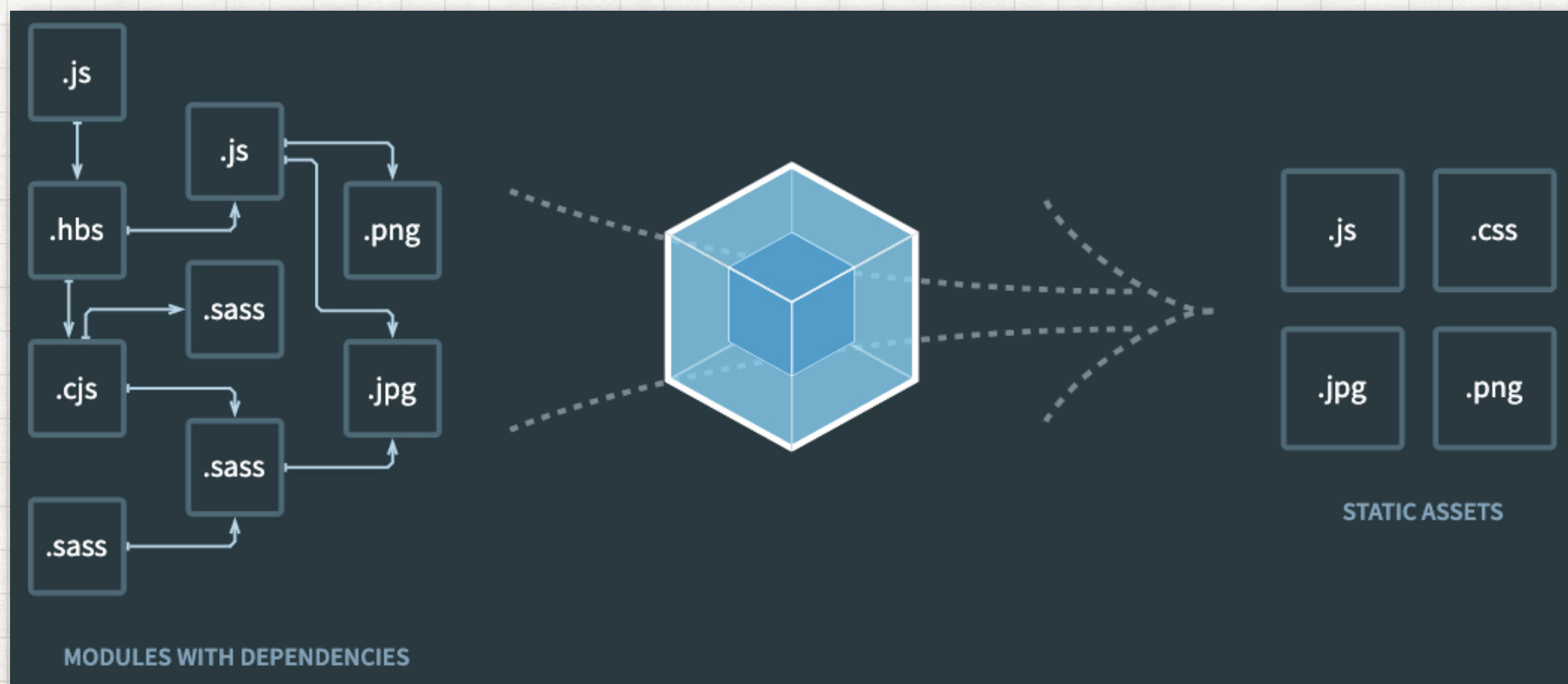
JS 输入.js

```
1 import React from 'react'
2 import ReactDOM from 'react-dom'
```

JS 输出.js

```
1 "use strict";
2
3 var _react = _interopRequireDefault(require("react"));
4
5 var _reactDom = _interopRequireDefault(require("react-dom"));
6
7 function _interopRequireDefault(obj) { return obj && obj.__esModule ? obj : { "default": obj }; }
```


JS 应用模块打包器



核心概念

- Entry
- Output
- Loader
- Plugin

ENTRY

```
untitled
1 module.exports = {
2   ...
3   entry: './src/index.js',
4   ...
5 };
```


OUTPUT

```
untitled
1 module.exports = {
2   ...
3   output: {
4     path: path.resolve(__dirname, "./build")
5   },
6   ...
7 };
```


LOADER

```
untitled
1  module.exports = {
2    ...
3    module: {
4      rules: [
5        {
6          test: /\.js$/,
7          exclude: /node_modules/,
8          use: ["babel-loader"]
9        },
10       {
11         test: /\.css$/,
12         use: ["style-loader", "css-loader"]
13       },
14       {
15         test: /\.txt$/,
16         loader: "raw-loader"
17       }
18     ]
19   },
20   ...
```


LOADER 例子

```
untitled
1 export default function rawLoader(source) {
2   const json = JSON.stringify(source)
3     .replace(/\u2028/g, '\\')
4     .replace(/\u2029/g, '\\');
5
6   return `export default ${json}`;
7 }
8
```


PLUGIN

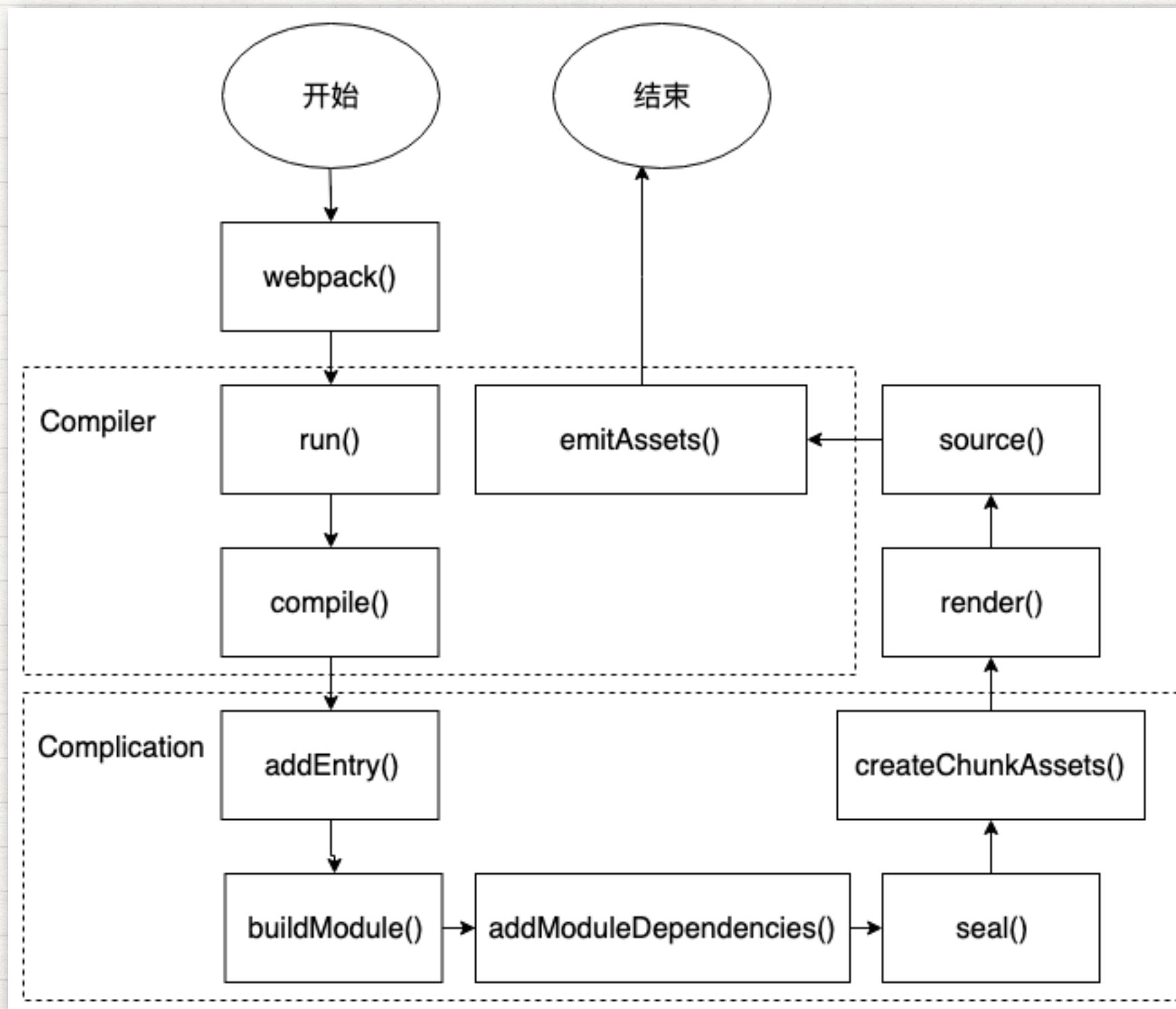
```
untitled
1  module.exports = {
2      ...
3      plugins: [
4          new HtmlWebpackPlugin({
5              title: "My Demo App",
6              template: "src/index.html"
7          })
8      ],
9      ...
10 };
```


PLUGIN 例子

JS beautify-plugin.js

```
1 class CodeBeautify {
2   apply(compiler) {
3     compiler.hooks.emit.tap("CodeBeautify", compilation => {
4       debugger;
5       Object.keys(compilation.assets).forEach(data => {
6         const oldContent = compilation.assets[data].source();
7         const newContent = oldContent.replace(/\\/\\*\\*\\*\\*\\*\\*\\/g, "");
8         compilation.assets[data] = {
9           source() {
10             return newContent;
11           },
12           size() {
13             return newContent.length;
14           }
15         };
16       });
17     });
18   }
19 }
20 module.exports = CodeBeautify;
```


核心流程



一些未涉及的点

- AST 如何解析和重新生成代码?
- Babel 和 Babel ployfill 的区别?
- webpack 依赖图如何生成?
- webpack tree-shaking、分包、模块热替换、本地调试?

附录

- demo 工程: <https://git.xiaojukeji.com/linyibing/babel-webapck-demo>

谢谢！